Guia de Relatorios

Atenção

- **Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.**
- **O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.**
- **Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.**

- **Aluno: Rafael Carrilho (rac4) — CESAR School | Repositório: github.com/RafaCarrilho/sync-car | Entrega: 04/11/2026**

1. Objetivo (3–5 linhas)

Implementar sync_car.h/sync_car.c: funções de sincronização para embarque ordenado de passageiros em um carro de metrô (METROREC), usando apenas pthread_mutex + pthread_cond (sem semáforo), um único lock por struct station, sem busy-wait. O código permite múltiplos passageiros retornarem de station_wait_for_car antes de qualquer um chamar station_on_board. Entrega validada pelo harness oficial sync_car-runner.c (não editável), com 100 passageiros concorrentes e múltiplos carros de capacidade aleatória.

2. Checklist de Requisitos

Requisito do enunciado	Status	Como testar / evidência
struct station definida em sync_car.c/.h	OK	sync_car.h
station_init	OK	compila, gcc -c limpo
station_wait_for_car	OK	teste integrado completo (seção 6)
station_on_board	OK	teste integrado completo (seção 6)
station_load_car	OK	teste integrado completo (seção 6)
Apenas pthread_mutex/pthread_cond, sem semáforo	OK	revisão de código
Único lock por station	OK	struct station tem 1 pthread_mutex_t
Múltiplos passageiros esperando simultaneamente, antes de qualquer embarque	OK	teste com sync_car-runner.c (100 passageiros, vários carros) — "Looks good!"
Sem busy-wait	OK	revisão de código (só cond_wait)
Makefile gerando sync_car	OK	make clean && make — só warning esperado (arquivo do professor)
README	OK	README.md no repo
Relatório em PDF (formato do Guia)	OK	este documento

3. Como Reproduzir

Compilar:

```
git clone https://github.com/RafaCarrilho/sync-car.git
cd sync-car
make
```

Executar (exemplo):

```
./sync_car
```

Saída esperada: várias linhas "car entering station..."/"car departed station..." terminando em "Looks good!".

Ambiente: WSL (Ubuntu). Git Bash/MINGW64 foi descartado — alarm()/SIGALRM (usados pelo harness para detectar travamento) não são confiáveis em Windows nativo, e o enunciado exige Linux/Unix/macOS.

4. Arquitetura (resumida)

Arquivos e responsabilidades:

- **sync_car.h** → struct station + protótipos das 4 funções
- **sync_car.c** → implementação completa das 4 funções

- **sync_car-runner.c** → harness oficial de teste (não editável)
- **Makefile** → build (make / make clean)
- **README.md** → descrição do projeto, build, execução, funções
- **evidencias.log** → log bruto de terminal (script -a, vai no .tar)

Decisões técnicas:

- Usei 2 condition variables pra não misturar as duas "personas" (passageiro e carro) e ajudar na hora de mandar o signal pra chegar no canto certo. Dessa forma consegui acordar de maneira adequada.
- Fiquei em dúvida se tirava ou não a flag de carro_presente, mas acabei optando por manter pois me ajudava mais no raciocínio e acho que seria mais perto do que eu implementaria na vida real.
- Usei signal (não broadcast) pra acordar o carro porque era desperdício usar o broadcast, que acorda todos, se eu estou alvejando apenas 1.

5. Estratégias e Diário de Desenvolvimento

Isto é o núcleo do relatório.

5.1 Estratégias da semana

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Contadores + 2 condvars	Abordagem única do início ao fim	— (não foi preciso mudar de estratégia)

5.2 Diário de Tentativas

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Desenhar struct station (3 contadores, 1 mutex, 2 condvars)	OK — justificada antes de codar	—	03/11 21:25	conversa
2	S1	1ª versão de station_init (parâmetro por valor, cond_passageiros inicializada 2x)	Erro, não commitado	Confundiu assinatura (cópia vs. ponteiro); copy-paste sem trocar nome da 2ª cond_init	03/11 21:37	revisão manual
3	S1	Corrigido: station_init(struct station *station), ->, 2º cond_init → cond_carro	OK — compila sem warnings	—	03/11 21:39	evidencias.log
4	S1	Compilar sync_car.c+sync_car-runner.c no WSL (só station_init)	Esperado — erro de link nas 3 funções não implementadas; station_init linka OK	Funções ainda não implementadas	03/11 22:02	evidencias.log (gcc -pthread -Wall -Wextra)
5	S1	Criação de sync_car.h	OK	Baseado no padrão de sync_car.h usado nas implementações	03/11 21:39	sync_car.h no repo

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
				anteriores (include guards, protótipos); struct station é autoria própria.		
6	S1	1ª versão de station_wait_for_car: só o while+cond_wait, sem lock	Erro, não testado	Faltava pthread_mutex_lock antes do while — cond_wait exige mutex já travado; UB	03/11 22:24	revisão manual
7	S1	2ª versão: lock/unlock e esperando++/--, embarcando++ certos, mas sem decrementar vagas	Erro, não testado	Sem vagas--, múltiplos passageiros poderiam reservar o mesmo assento (over-booking)	03/11 22:30	revisão manual
8	S1	3ª versão: adicionado vagas-- dentro da seção crítica, antes do unlock	OK	—	03/11 22:31	revisão manual + trace manual (cenário Ana/Bruno, 1 vaga)
9	S1	Compilação limpa de station_init+station_wait_for_car no WSL	OK — só os 2 erros de link esperados (station_load_car, station_on_board)	—	03/11 22:37	evidencias.log
10	S1	Commit "Implementa station_wait_for_car"	OK	—	03/11 22:40	git log
11	S1	station_on_board: lock, embarcando--, unlock — sem o signal (deixado como comentário)	Incompleto, não testado	Sabia que precisava de signal mas não lembrava qual condvar nem o porquê de signal vs broadcast	03/11 22:43	revisão manual
12	S1	Completo: pthread_cond_signal(&station->cond_carro) antes do unlock	OK	—	03/11 22:44	revisão manual
13	S1	1ª versão de station_load_car: seta carro_presente, broadcast, espera condicional, zera carro_presente no fim — sem station->vagas =	Bug reproduzido de propósito, print capturado	vagas nunca atualizado pro carro novo; condição de saída usa vagas==0 no ||, fica sempre verdadeira — retorna instantâneo. Runner acusa "returned early!". Também	03/11 22:53	evidencias.log + print (bug principal)

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
		count		trava station_wait_for_car (vagas==0 pra sempre)		
14	S1	Corrigido: station->vagas = count; logo após carro_presente = 1	OK — teste completo com 100 passageiros passou, "Looks good!"	—	03/11 22:57	evidencias.log + print (validação final)
15	S1	Makefile (padrão das implementações anteriores)	OK — make clean && make compila limpo	—	04/11	evidencias.log
16	S1	README.md (descrição, build, execução, funções, arquivos, ambiente)	OK — commitado e renderizando no GitHub	—	04/11 21:04	github.com/RafaCarrilho/sync-car
17	S1	Validação extra: -fsanitize=thread + 15 execuções consecutivas, vagas variando entre 1 e 49 por carro	OK — todas "Looks good!". TSan reportou races só em sync_car-runner.c (globais do harness, não editável) — nenhuma em struct station/mutex/condvar	—	04/11 21:15	logs do TSan (run_1 a run_15)

6. Evidências

6.1 Log automático (faz o trabalho pesado por você)

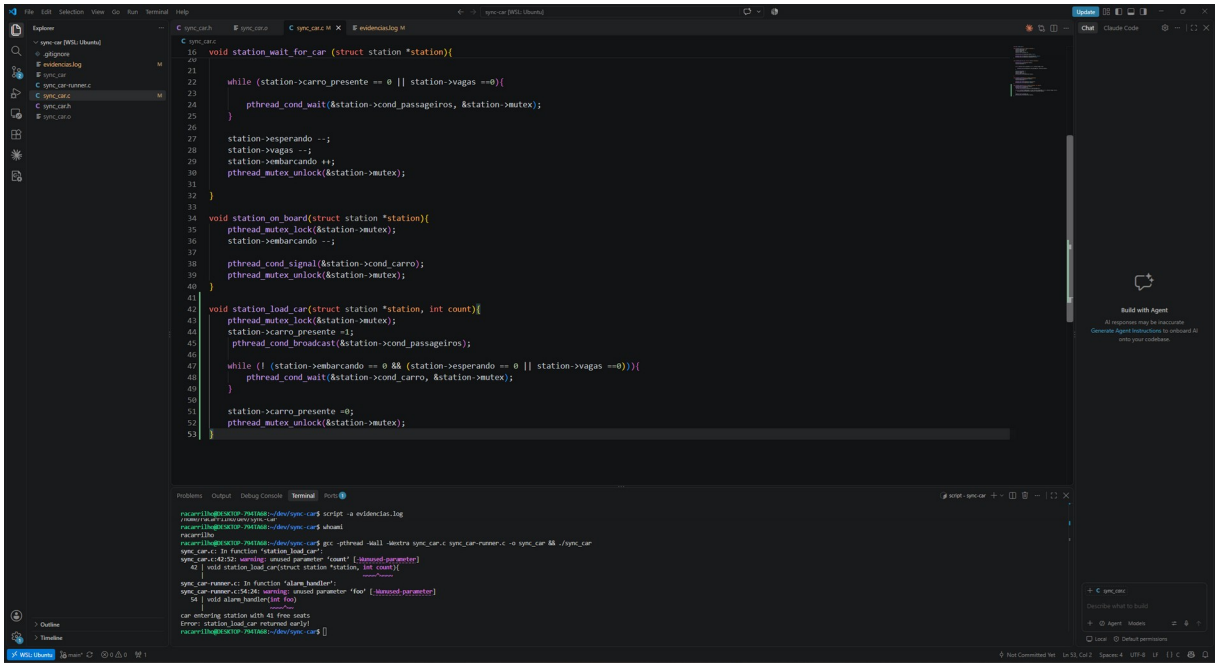
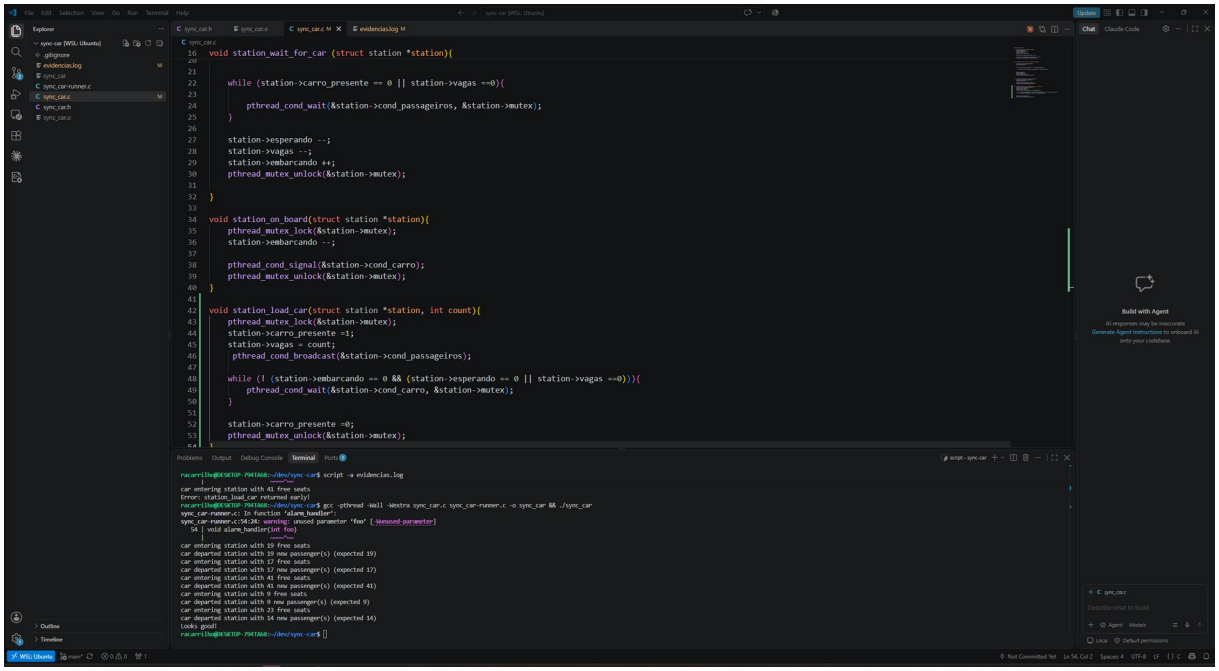
No início da sessão de trabalho, rode:

```
script -a evidencias.log
```

Log rodando no WSL a partir de 03/11 22:02 (tentativa anterior em Git Bash foi descartada por mudança de ambiente; houve um gap de ~30min por travamento do PC, sem perda de trabalho; religado normalmente antes dos comandos finais de 04/11). Contém a reprodução do bug #13 do Diário e a execução completa e bem-sucedida da correção #14. O evidencias.log vai dentro do .tar final.

6.2 Prints

- Erro principal: o momento do bug mais relevante da semana.
- Validação final: teste passando / saída correta.

- 
- 

6.3 Validação com ThreadSanitizer

15 execuções do teste oficial, compiladas com -fsanitize=thread, vagas por carro variando entre 1 e 49. Todas terminaram em "Looks good!". O TSan sinalizou data races em todas as rodadas, mas 100% delas localizadas em sync_car-runner.c (globais de controle do próprio harness — threads_completed e load_car_returned —, não protegidos por lock; comportamento do arquivo do professor, fora do escopo da entrega). Nenhuma race tocou struct station, mutex ou condvar de sync_car.c.

7. Uso de IA

Onde usei IA (2–3 linhas):

Claude como tutor principal (sessão completa, método socrático — nunca escreveu código de sync_car.c/.h). ChatGPT consultado em paralelo no início da sessão para uma explicação com didática alternativa; descontinuado para evitar divergência/retrabalho. Um erro da IA foi identificado e corrigido.

Prompts principais (liste, sem história):

- "Pra que 2 cond variable???"
- "signal é 1on1 e o broadcast é geral. de que adianta sinalar o carro pra acordar no momento da reserva? quando a condicao de embarcando ainda ta certamente negativa. no ultimo station on board que a condicao

vai ser cumprida"
- "nao tem carro e carro cheio ambos servem pra nada."
- "Pera... era no WSL não era? Carai e meu evidencias o que eu faço????"
- "Vou pedir pro GPT me explicar a mesma coisa que tu explicou só que melhor de didática, não pedirei código"
- "roda nossa entrega contra criterios de avaliação e ve se tem algum erro que o teste nao pega no nosso código"

O que validei manualmente e como:
Releitura do enunciado oficial (PDF) para confirmar a condição exata de retorno de station_load_car; compilação e teste de link no WSL; trace manual de station_wait_for_car com 2 threads concorrentes; reprodução do bug de vagas não inicializada seguida da execução completa do teste oficial (100 passageiros) após a correção; build via Makefile; leitura dos relatórios do ThreadSanitizer confirmando que nenhuma race tocava a implementação.

8. Reflexão Final (5–8 linhas)

Reflexão:

O que me deu mais trabalho foi entender qual era o papel de cada função, no sentido de quem deve acordar quem. Por que precisar dar lock ou não, demorei a entender isso também. O que me ajudou a entender o lock foi uma brincadeira americana do "talking stick" e lembrar do Pint-OS, de quando a gente precisava ligar interrupts off em algumas partes. Travar o mutex me lembrou disso.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	Sim
Rodei os testes e evidencias.log foi gerado	Sim
Tenho prints obrigatórios	Sim
Testei pelo menos um caso limite e um caso inválido	Parcial — testei casos limite (station_load_car com 0 vagas, antes e depois de passageiros esperarem); não há caso inválido aplicável às assinaturas das funções
Preenchi a seção de uso de IA	Sim
Revisei o relatório e removi frases genéricas / vazias	Sim

10. Se eu tivesse mais 2 horas

Se eu tivesse mais duas horas eu estudaria mais o código base para entender como eu faria se fosse eu mesmo. Teve muita coisa que eu não entendi e precisei decifrar ao longo da implementação, o que foi um desafio legal. Mas, com mais tempo, eu não mudaria a minha entrega. Tive tempo o bastante para isso.

Regras de Integridade (não mudam)

- Ocorrência relevante sem evidência (log ou print) = não conta para a nota.
- Contradição entre prints, log e relato = relatório perde credibilidade.
- Indício de evidência fabricada = entrega invalidada.